

ME315 Lab02

Letícia Santos Tavares

HANDS ON

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)?

Resposta: Para determinar o percentual de vôos atrasados na chegada em mais de 10 minutos, são necessárias 2 estatísticas: o número total de vôos atrasados na chegada em mais de 10 minutos do dia, ou seja, $ARRIVAL_DELAY > 10$ e o número total de vôos ocorridos no dia. Isto para cada dia do mês e para cada mês do ano, para as companhias AA, DL, UA e US. Dessa forma, o percentual pode ser calculado pela proporção **% de vôos atrasados** = $(ARRIVAL_DELAY > 10) / \text{total de vôos do dia}$

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e. A função deve receber apenas dois argumentos:
 - I) `input`: o conjunto de dados (referente ao lote em questão);
 - II) `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
getStats <- function(input, pos) {  
  input |>  
  filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |>  
  filter(  
    !is.na(YEAR),  
    !is.na(MONTH),  
    !is.na(DAY),  
    !is.na(AIRLINE),  
    !is.na(ARRIVAL_DELAY)  
  ) |>  
  group_by(YEAR, MONTH, DAY, AIRLINE) |>  
  summarise(  
    numero_voos = n(),  
    voos_atrasados = sum(ARRIVAL_DELAY > 10),  
    .groups = "drop"  
  )  
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
library(dplyr)  
library(readr)  
dados_parciais <- read_csv_chunked(  
  file = "flights.csv.zip",  
  callback = DataFrameCallback$new(getStats),  
  chunk_size = 100000,  
  col_types = cols_only(  
    YEAR = col_integer(),
```

```

    MONTH = col_integer(),
    DAY = col_integer(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

4. Crie uma função chamada `ComputeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
 - i) **Cia**: sigla da companhia aérea;
 - ii) **Data**: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii) **Perc**: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```

library(dplyr)

computeStats <- function(dados) {
  dados |>
    group_by(AIRLINE, YEAR, MONTH, DAY) |>
    summarise(
      total_voos = sum(numero_voos),
      total_atrasados = sum(voos_atrasados),
      .groups = "drop"
    ) |>
    transmute(
      Cia = AIRLINE,
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
      Perc = total_atrasados / total_voos
    )
}

dados_interesse <- computeStats(dados_parciais)

```

```
install.packages("remotes")
```

Installing package into '/home/leticia/R/x86_64-pc-linux-gnu-library/4.5'
(as 'lib' is unspecified)

```
remotes::install_github("jayjacobs/ggcal")
```

Skipping install of 'ggcal' from a github remote, the SHA1 (ab1a85ad) has not changed since 1
Use `force = TRUE` to force installation

```
library(ggcal)
library(ggplot2)

pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

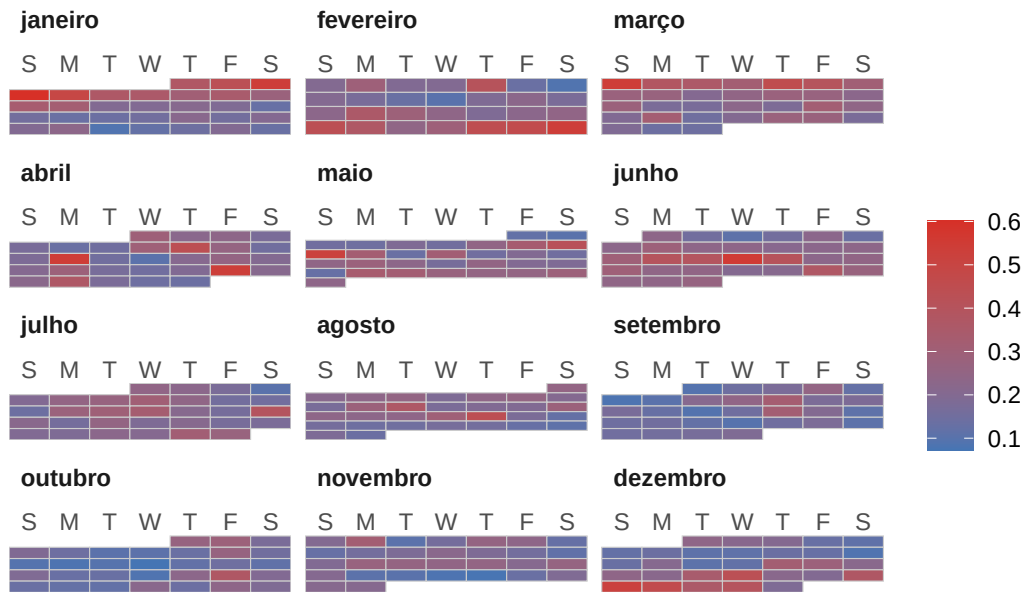
baseCalendario <- function(stats, cia) {
  dados_cia <- stats |>
    filter(Cia == cia)

  ggcal(dados_cia$Data, dados_cia$Perc)
}

cAA <- baseCalendario(dados_interesse, "AA")
cDL <- baseCalendario(dados_interesse, "DL")
cUA <- baseCalendario(dados_interesse, "UA")
cUS <- baseCalendario(dados_interesse, "US")

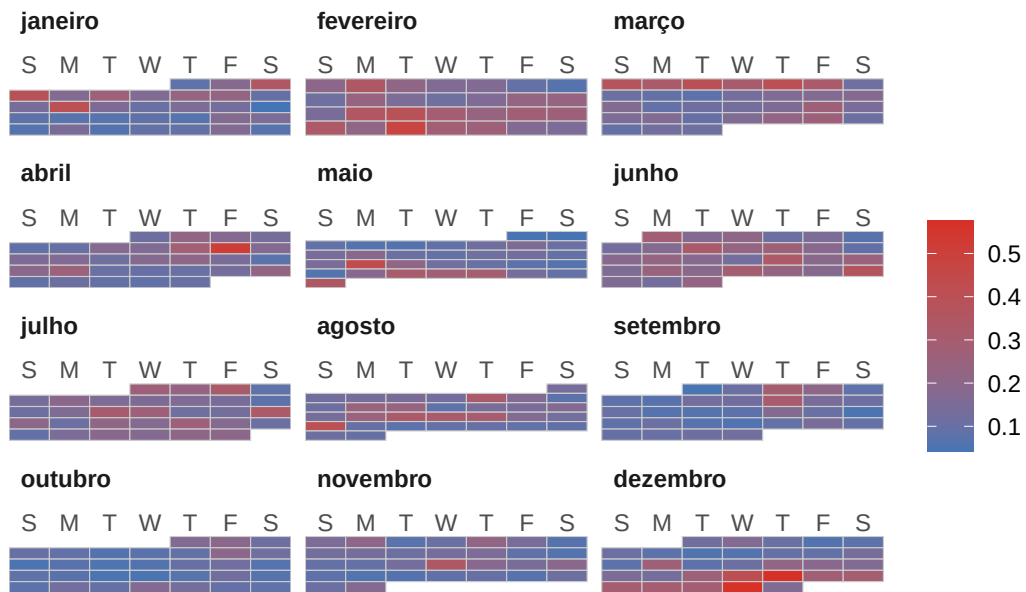
cAA + pal + ggtitle("AA")
```

AA



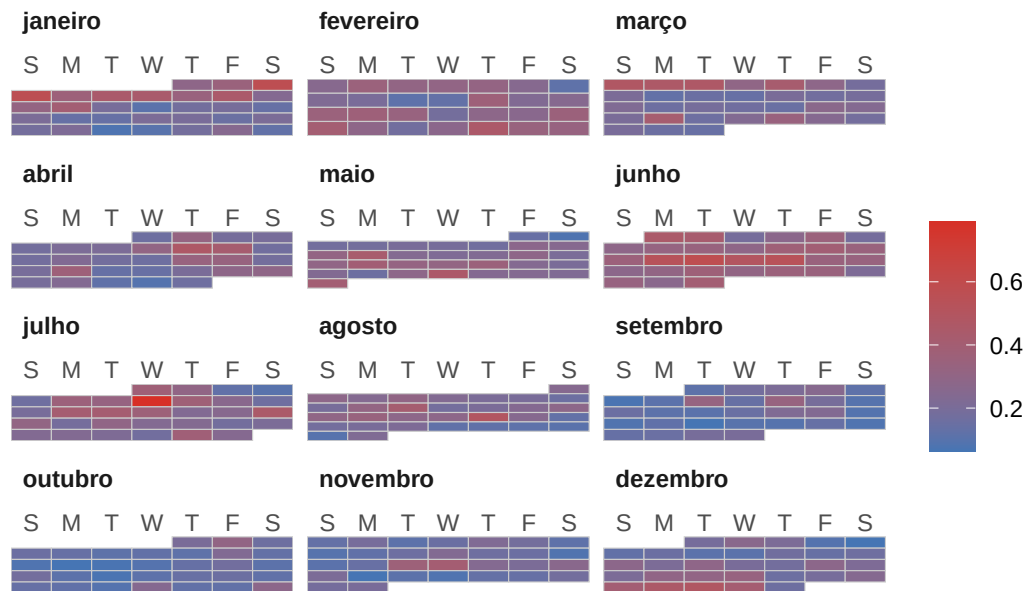
```
cDL + pal + ggtitle("DL")
```

DL



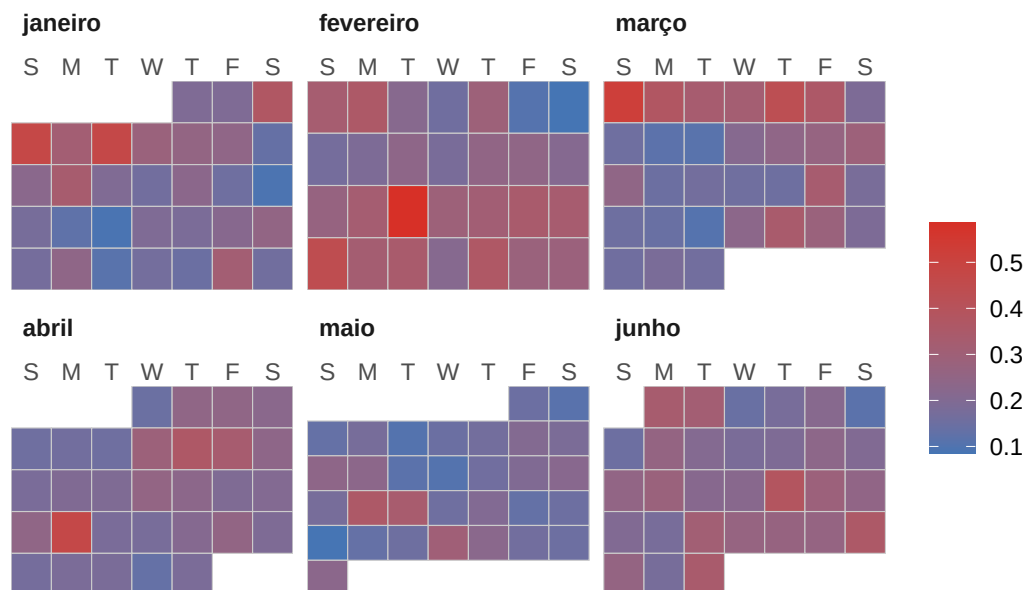
```
cUA + pal + ggtitle("UA")
```

UA



```
cUS + pal + ggtitle("US")
```

US



```
Sys.setenv(TZ= "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 19:50:39 -03"
```